Raspberry Pi Telemetry Host

System Overview

Authored by: Ryan Tengan

Table of Contents

Revision History

Version	Date	Author	Comments
1	10/20/2025	Ryan Tengan	Initial Document
2	3/10/2026	Ryan Tengan	Added Telemetry section

Introduction

Background

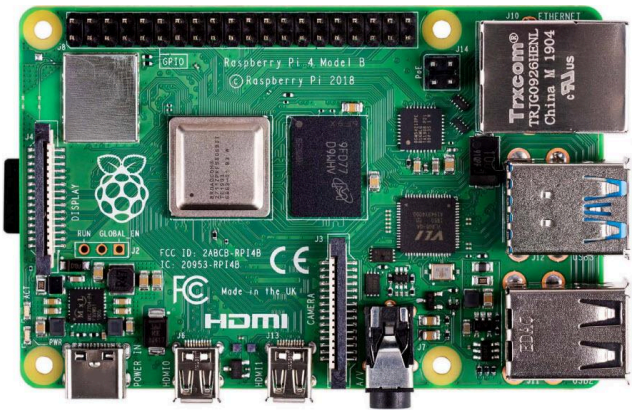

Figure 1: Raspberry Pi Model 4B

In previous years, the FRUCD Firmware subteam used a single “Dashboard” PCB — now referred to as the *Vehicle Control Unit (VCU)* — to host the driver dashboard and vehicle state determination firmware. Later in the FE12 development year (2024–2025), a hardware issue was found with the dashboard circuitry on the VCU, resulting in conflicts with surrounding safety-critical components. To decouple from safety-critical firmware, the dashboard and other related functionalities are now handled by the *Raspberry Pi Telemetry Host* (Figure 1).

Goals

The three primary components of the Raspberry Pi Telemetry Host are as follows:

- **Telemetry:** Integrating data analytics software to remotely monitor vehicle performance during or after test runs.
- **Dashboard:** Display real-time, vehicle information to the driver, emphasizing easy readability and minimal latency updates.

The rest of this document is written to provide a general overview of these components.

Telemetry

Overview

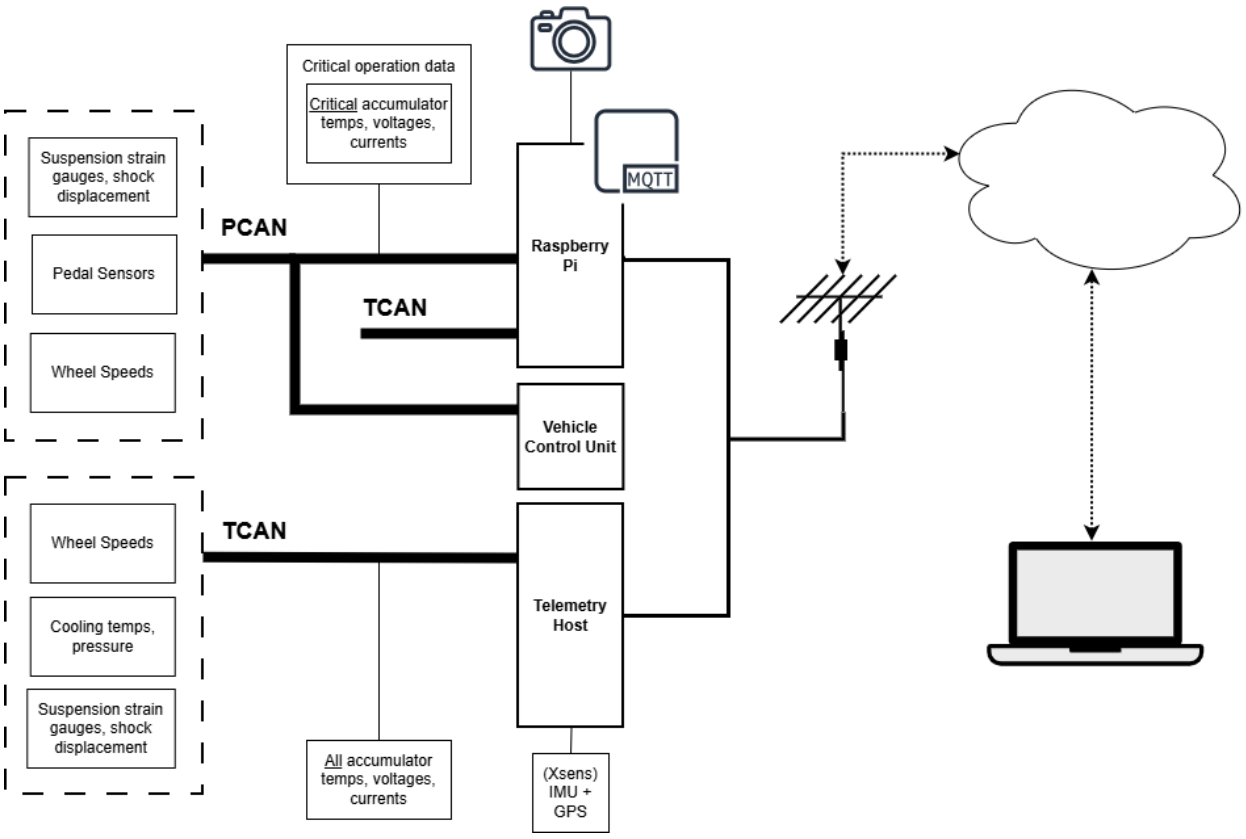

Figure 1: Telemetry System

As shown in Figure 1, the Raspberry Pi functions as the central telematics control unit (TCU), serving vehicle data received from the following:

- **Controller Area Network (CAN) buses:** Safety-critical data required for vehicle operation, as well as non-critical data used for performance analysis.
- **Web camera:** Records driver performance during test drives.
- **Telemetry Host:** Receives inertial measurement unit (IMU) and global navigation satellite system (GNSS) data from the Xsens MTi-670 via an Ethernet connection.

Connected to a router, the Raspberry Pi is then accessible remotely through a VPN connection.

Components

2-Channel CAN HAT

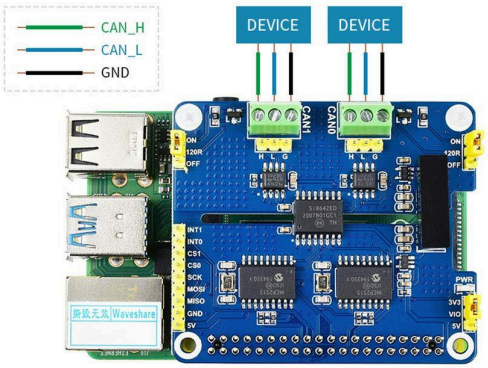

Figure 2: 2-CH CAN HAT

Our vehicle telemetry system utilizes two separate CAN buses to reduce overall network traffic and improve communication reliability:

- Powertrain CAN (PCAN): Transmits data critical to vehicle operation and safety.
- Telemetry CAN (TCAN): Carries non-critical data used for performance analysis and post-run evaluation.

To enable parallel communication with both PCAN and TCAN, a 2-Channel CAN HAT (shown in Figure 2) is connected to the Raspberry Pi via its GPIO header. The HAT integrates onboard MCP2515 CAN controllers that interface with the Raspberry Pi over SPI. CAN frames are transmitted and received through the Linux SocketCAN framework, allowing the system to process CAN traffic using standard network interface abstractions.

Message Queuing Telemetry Transport (MQTT)

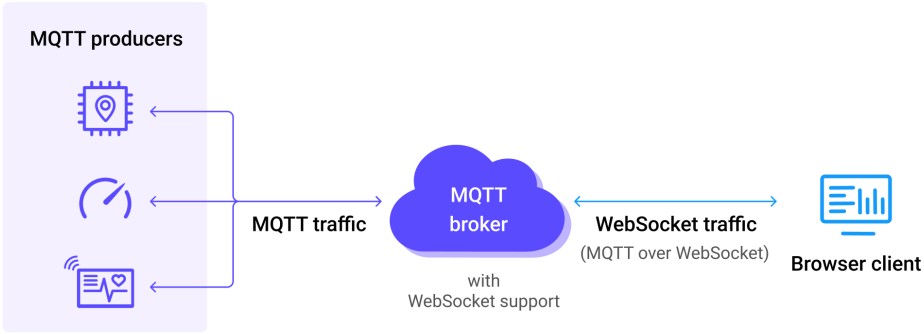

Figure 3: MQTT over WebSockets

In order to publish decoded CAN bus data to clients, the Raspberry Pi uses an MQTT broker. Benefits of the MQTT broker include but are not limited to:

- Real-time data distribution: Clients receive updates asynchronously without polling, reducing latency and processing overhead.
- Low bandwidth overhead: The lightweight publish/subscribe architecture minimizes network traffic, making it well-suited for embedded telemetry systems.

As demonstrated in Figure 3, our team then accesses the MQTT broker through a WebSocket connection, allowing for easily-accessible data monitoring through a web application.

Telemetry Web App

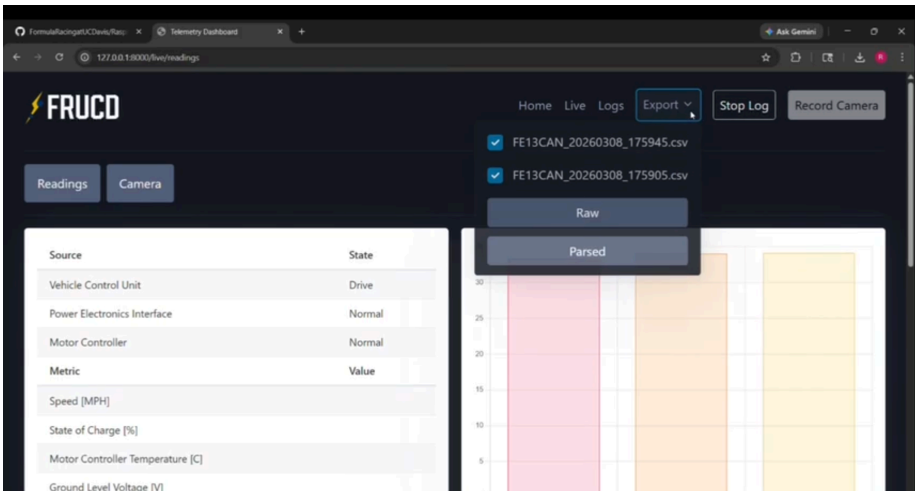

Figure 4: Telemetry Web App

The Telemetry Web App (shown in Figure 4) is a single-page web application served by the Raspberry Pi, allowing clients to access live vehicle health data, analyze data logs, and view a live camera stream. This is a user-facing application, meant for use by all subteams, and should be developed accordingly.

Design Considerations

Data Acquisition

When writing data acquisition software, use high-performance programming languages such C++ or Rust. These languages provide fine-grained control over memory management, efficient interrupt handling, and direct interaction with device drivers, which are all critical for maintaining low latency and preventing data loss in high-throughput systems.

Dashboard

Overview

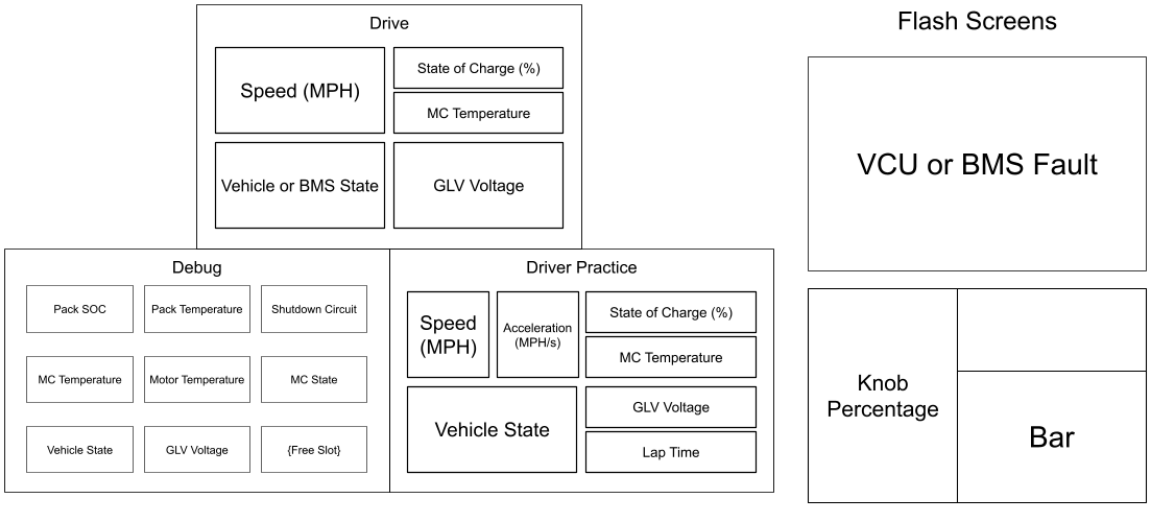

Figure 5: Dashboard

The dashboard displays live vehicle information to the driver. As shown in Figure 5, FRUCD's dashboard adds extra features for enhanced driver feedback. This includes pages such as *Debug Mode*, *Driver Practice Mode*, fault screens, and bar gauges. The different modes depend on driver input, whereas the flashscreens notify of critical changes in vehicle conditions.

Components

Drive Mode

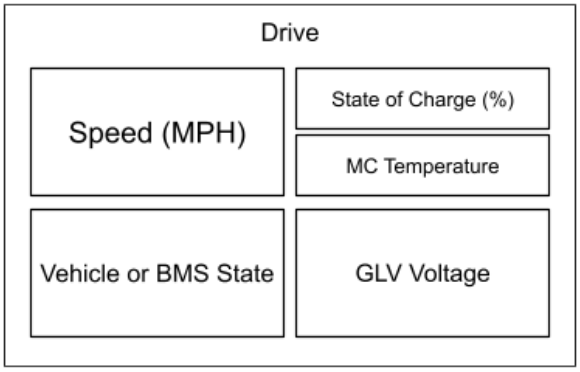

Figure 6: Drive Mode Layout

Drive Mode (Figure 6) is the default mode when entering Startup and displays the following parameters:

Parameter	Units	Source	Description	Color
Speed	mph	VCU	$speed\ [mph] = \frac{Wheel\ RPM \times Tire\ circumference \times 60}{63360}$	blue
		MC	$speed\ [mph] = \frac{Motor\ RPM \times 60\pi \times Tire\ diameter}{Final\ Drive\ Ratio \times 63360}$	blue
State	N/A	VCU	Operating state	green
		VCU	“BSPD tripped” or “Uncalibrated” faults	yellow
		BMS, VCU	Other faults (BMS prioritized)	red
SOC	%	PEI	State of Charge	red
MC Temp	C	MC	Max temperature of MC modules (A, B, C) < 45 C	green
			Max temperature of MC modules (A, B, C) < 50 C	yellow
			Max temperature of MC modules (A, B, C) >= 50 C	red
GLV V	V	MC	Ground Level Voltage > 10 V	green

			Ground Level Voltage > 9 V	yellow
			Ground Level Voltage <= 9 V	red

Debug Mode

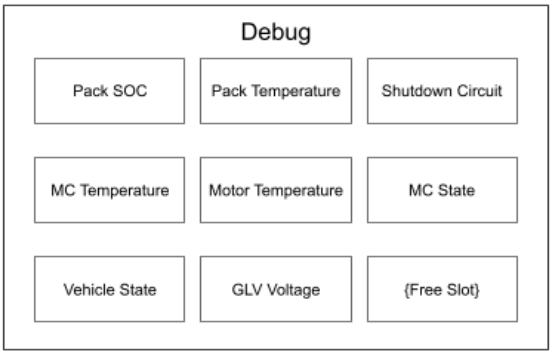

Figure 7: Debug Mode Layout

Debug Mode (Figure 7) assists with stationary debugging. Beyond what is already displayed in Drive Mode, Debug Mode displays the following:

Metric	Source	Description
Shutdown Circuit	PEI	Current shutdown circuit state (which switch is flipped)
MC Fault	MC	Motor controller fault determination
Free Slot	N/A	Temporary placeholder for any desired data

Driver Practice Mode

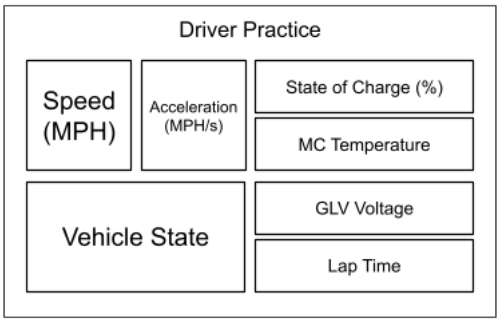

Figure 8: Driver Practice Mode Layout

Driver Practice Mode (Figure 8) serves to help drivers improve their skills by displaying GPS and IMU data.

Metric	Source	Description
Acceleration	Xsens	Offered by the Xsens IMU
Lap Time	Xsens	Calculated using Xsens GPS data

Flash Screens

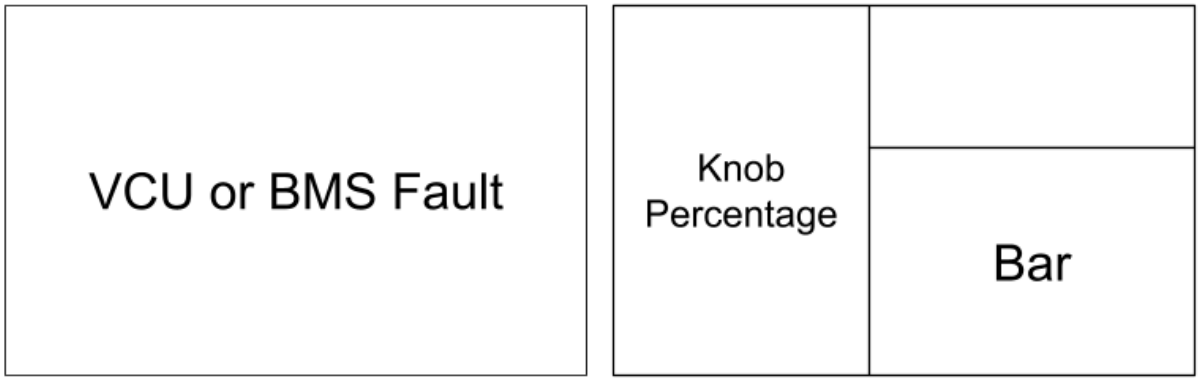

Figure 9: Fault Screen and Bar Gauge Layout

Figure 9 displays the dashboard's two primary flash screens. The fault screen temporarily flashes when the vehicle enters a VCU or BMS fault (prioritizing BMS faults), whereas the bar gauge displays the knob turn percentage corresponding to driver input.

Parameter	Units	Source	Description	Color
State	N/A	BMS, VCU	Temporary flashscreen when entering a fault state	red
Knob 1	%	VCU	Torque limit	red
Knob 2	%	VCU	Launch control parameter	blue

Design Considerations

Performance

Choosing the right programming language and GUI library will greatly influence UI rendering speed, which is especially important for the dashboard where performance is critical. Programming languages like C or C++ are preferred.

User Interface

Prioritize readability by using high-contrast colors and remaining fairly consistent with previous years' layouts. Keep in mind that environmental conditions already affect dashboard visibility, such as wearing the helmet or glare over the display.